`common_reference_t` of `reference_wrapper` Should Be a Reference Type

Document #:	P2655R0
Date:	2022-09-12
Project:	Programming Language C++
Audience:	SG9, LEWG

Reply-to:	Hui Xie <hui.xie1990@gmail.com> S. Levent Yilmaz <levent.yilmaz@gmail.com>

Contents

— 1 Revision History

  — 1.1 R0

— 2 Abstract

— 3 Motivation and Examples

— 4 Implementation Experience

— 5 Wording

— 6 References

§ 1 Revision History

§ 1.1 R0

— Initial revision.

§ 2 Abstract

This paper proposes a fix that makes the `common_reference_t<T&, reference_wrapper<T>>` a reference type `T&`.

§ 3 Motivation and Examples

C++20 introduced the meta-programming utility `common_reference` 21.3.8.7 [meta.trans.other] in order to programmatically determine a common reference type to which one or more types can be converted or bounded.

The precise rules are rather convoluted, but roughly speaking, for given two non-reference types `X` and `Y`, `common_reference<X&, Y&>` is equivalent to the expression `decltype(false ? declval<X&>() : declval<Y&>())` provided it is valid. And if not, then user or a library is free to specialize the `basic_common_reference` trait for any given type(s). (Two such specializations are provided by the standard

library, namely, for `std::pair` and `std::tuple` which map `common_reference` to their respective elements.) And if no such specialization exists, then the result is `common_type<X,Y>`.

The canonical use of `reference_wrapper<T>` is its being a surrogate for `T&`. So it might be surprising to find out the following:

```
int i = 1, j = 2;
std::reference_wrapper<int> jr = j; // ok - implicit constructor
int & ir = std::ref(i); // ok - implicit conversion
int & r = false ? i : std::ref(j); // error!
```

The reason for the error is not because `i` and `ref(j)` (an `int&` and a `reference_wrapper<int>`) are incompatible. It is because they are too compatible! Both types can be converted to one another, so the type of the ternary expression is ambiguous.

Hence, per the current rules of `common_reference`, the evaluation falls back to `common_type<T, reference_wrapper<T>>`, whose `::type` is valid and equal to `T` (there is no ambiguity here with prvalue `T` and `reference_wrapper<T>`, since former is convertible to latter, but not vice versa).

The authors believe the current determination logic for `common_reference` for an lvalue reference to a type `T` and its `reference_wrapper<T>` is merely an accident, and is incompatible with the canonical purpose of `reference_wrapper`. Therefore, this article proposes an update to the standard which would change the behavior of `common_reference` to evaluate as `T&` given `T&` and any reference or prvalue of type `reference_wrapper<T>`, commutatively. Furthermore, the authors propose to implement this change via a partial specialization of `basic_common_reference` trait.

Below are some motivating examples:

C++20	Proposed

`static_assert(same_as<`
`  common_reference_t<int&,`
`  reference_wrapper<int>>,`
`  int>);`
 	
`static_assert(same_as<`
`  common_reference_t<int&,`
`  reference_wrapper<int>&>,`
`  int>);`
	`static_assert(same_as<`
	`  common_reference_t<int&,`
	`  reference_wrapper<int>>,`
	`  int&>);`
	
	`static_assert(same_as<`
	`  common_reference_t<int&,`
	`  reference_wrapper<int>&>,`
	`  int&>);`

`class MyClass {`
`  vector<vector<Foo>> foos_;`
`  Foo delimiter_;`
 	
`  public:`
`  void f() {`
`  auto r = views::join_with(foos_,`
 	
`  views::single(std::ref(delimiter_)));`
`  for (auto&& foo : r) {`
`  // foo is a temporary copy`
`  }`
`  }`
`};`
	`class MyClass {`
	`  vector<vector<Foo>> foos_;`
	`  Foo delimiter_;`
	
	`  public:`
	`  void f() {`
	`  auto r = views::join_with(foos_,`
	
	`  views::single(std::ref(delimiter_)));`
	`  for (auto&& foo : r) {`
	`  // foo is a reference to the original element`
	`  }`
	`  }`
	`};`

`class MyClass {`
`  vector<Foo> foo1s_;`
`  Foo foo2_;`
	`class MyClass {`
	`  vector<Foo> foo1s_;`
	`  Foo foo2_;`

C++20	Proposed

`public:`
`  void f() {`
`  auto r = views::concat(foo1s_,`
`  views::single(std::ref(foo2_)));`
`  for (auto&& foo : r) {`
`  // foo is a temporary copy`
`  }`
`  }`
`};`
	`public:`
	`  void f() {`
	`  auto r = views::concat(foo1s_,`
	`  views::single(std::ref(foo2_)));`
	`  for (auto&& foo : r) {`
	`  // foo is a reference to the original element`
	`  }`
	`  }`
	`};`

In the second and the third example, the user would like to use `views::join_with` and `views::concat` [P2542R2], respectively, with a range of `Foo`s and a single `Foo` for which they use a `reference_wrapper` to avoid copies. Both of the range adaptors rely on `common_reference_t` in their respective implementations (and specifications). As a consequence, the counter-intuitive behavior manifests as shown, where the resultant views' reference type is a prvalue `Foo`. There does not seem to be any way for the range adaptor implementations to account for such use cases in isolation.

§ 4 Implementation Experience

— The authors implemented the proposed wording below without any issue.

— The authors also applied the proposed wording in LLVM's libc++ and all libc++ tests passed.

§ 5 Wording

Modify 22.10.2 [functional.syn] to add to the end of `reference_wrapper` section:

```
template<class T, template<class> class TQual, template<class> class UQual>
struct basic_common_reference<T, reference_wrapper<T>, TQual, UQual>;

template<class T, template<class> class TQual, template<class> class UQual>
struct basic_common_reference<reference_wrapper<T>, T, TQual, UQual>;
```

Add the following subclause to 22.10.6 [refwrap]:

§ 22.10.6.? `common_reference` related specialization [refwrap.common.ref]

```
template<class T, template<class> class TQual, template<class> class UQual>
struct basic_common_reference<T, reference_wrapper<T>, TQual, UQual> {
    using type = common_reference_t<TQual<T>, T&>;
};

template<class T, template<class> class TQual, template<class> class UQual>
struct basic_common_reference<reference_wrapper<T>, T, TQual, UQual> {
    using type = common_reference_t<UQual<T>, T&>;
};
```

§ 6 References

[P2542R2] Hui Xie, S. Levent Yilmaz. 2022-05-11. views::concat.
https://wg21.link/p2542r2